

Qwen3-Coder-Next Tier 3

Part 15 Detailed Implementation Specification

Near-Codex Integrated System

Engineering specification for the final integrated system that turns the earlier subsystem stack into one coherent, measurable, and deployable autonomous software engineering platform.

Document role	Implementation specification
Part	15
Subsystem	Near-Codex Integrated System
Dependency posture	Consumes all prior parts; produces the final integrated operating envelope

Scope note: this document does not redesign architecture. It defines the engineering work required to turn already-defined parts into a single integrated system with stable contracts, measurable behavior, and release-grade controls.

1. Purpose

Part 15 exists to integrate the entire stack into a single operational system. The earlier parts each solve a real problem, but none of them is sufficient alone. A planner without execution is theory. A memory system without retrieval discipline is clutter. A benchmark harness without release gates is a museum label. The integrated system is the point at which all of those pieces have to agree on the same task, the same state, the same evidence, and the same outcome.

This part solves the system-level problem of coherence. It specifies how tasks enter, how they flow through planning, research, coding, testing, review, evaluation, recovery, repository intelligence, context compression, benchmarking, and deployment, and how the system returns a result that can be trusted by an engineer or an approval gate.

Future work depends on this part because it establishes the final end-to-end operating contract. It defines what a complete run looks like, which state transitions are legal, which artifacts must exist before a run can be considered complete, and which signals are sufficient to promote or block a change.

Design intent: Part 15 is not “more features.” It is integration discipline, control boundaries, and proof that the prior fourteen parts function as one system rather than fourteen impressive excuses.

2. Scope

Included:

- End-to-end task lifecycle from request ingestion to final result publication.
- Cross-part orchestration rules for planner, research, coding, testing, review, evaluation, recovery, memory, repository intelligence, context compression, and benchmark participation.
- System-level state model, event model, and artifact model.
- Release-grade control points for human approval, automated gates, and rollback signaling.
- Top-level observability, traceability, and failure reporting across the entire run.
- Completion criteria for a near-Codex operating mode that is measurable rather than rhetorical.

Excluded:

- Low-level implementation details for individual agents; those belong in their own part documents.
- Changes to the architecture diagram itself; architecture is already finalized.
- Training data pipeline design, model weights, or inference server internals.
- Business logic of external products under test.
- Deployment implementation specifics beyond the final approval and release contract.

Included	Excluded
End-to-end run contracts	Agent prompt engineering
Cross-part state flow	Model training pipelines
Approval and release gates	Production app business rules
System-level observability	Replacing subsystem specifications

3. System Overview

The integrated system takes a user task and turns it into a traceable execution sequence with evidence at every stage. The chain below is the operating path, not just a conceptual sketch.

```
User / CI / Scheduled Trigger
-> Task Intake
-> LangGraph Orchestrator
-> Planning
-> Research
-> Context Compression
-> Coding in Worktree
-> Testing
-> Review
-> Evaluation
-> Failure Recovery if needed
-> Benchmark Capture
-> Repository Intelligence Update
-> Human Approval Gate
-> Deployment / Publication
-> Final Run Record and Artifacts
```

The important property is that each stage publishes a typed result to a shared run context rather than mutating invisible global state. Every later stage can see what happened earlier, but only through a contract. That is what makes the system debuggable and auditable.

Conceptual example of one run:

```
Task: implement auth refresh flow
Plan: 5 steps
Research: 3 docs, 2 API surfaces, 1 repo graph query
Memory lookup: 2 prior fixes reused
Coding: 1 isolated worktree
Testing: 18 tests, 1 failure, 1 retry
Review: security concern flagged and patched
Evaluation: accepted
Benchmark: pass
Approval: human approved
Deployment: published
Outcome: complete with traceable artifacts
```

4. Internal Architecture

The integrated system consists of five control planes and one execution plane. The control planes exist to keep the execution plane honest.

Plane	Purpose	Primary responsibility
Task control plane	Owens ingestion, identity, and run creation	Convert an external request into a tracked run
Orchestration plane	Owens stage ordering and handoff semantics	Route tasks through planner, research, code, test, review, evaluate, and recover
Evidence plane	Owens state, memory, repository intelligence, and benchmark records	Make every decision explainable and replayable
Safety plane	Owens human approval, policy, and release gates	Prevent unsafe promotion or deployment
Observability plane	Owens telemetry, traces, and reports	Expose performance, regressions, and failure causes
Execution plane	Owens worktrees, tool calls, and artifacts	Perform the actual work under isolated conditions

Required top-level abstractions include RunEnvelope, StageResult, ArtifactRef, EvidenceBundle, GateDecision, and ReleaseOutcome. The system must be able to answer three questions at any time: what is the current run, what evidence supports the current state, and what prevents or allows the next transition?

Integration rule: control planes may veto, throttle, or redirect the execution plane; the execution plane may not override control decisions. This asymmetry is intentional. Systems that let execution rewrite policy are usually how teams end up shipping expensive confusion.

Reference interaction flow:

```
RunEnvelope
-> Orchestrator
-> Stage Dispatcher
    -> Planner
    -> Research
    -> Memory Retrieval
    -> Context Compression
    -> Coding / Worktree
    -> Testing
    -> Review
    -> Evaluation
    -> Recovery (if failure)
    -> Benchmark Event Publish
    -> Approval Gate
    -> Release Manager
-> Final Evidence Bundle
```

5. Project Structure

The integrated system should be organized by operational responsibility, not by the historical order in which modules were written.

Directory	Purpose
integrated/	Root package for end-to-end orchestration and shared runtime utilities.
integrated/runtime/	RunEnvelope creation, lifecycle management, and stage routing.
integrated/contracts/	Shared typed contracts for run state, stage results, events, and artifacts.
integrated/policy/	Approval policy, release policy, safety policy, and gating rules.
integrated/evidence/	Evidence aggregation, provenance, run records, and audit bundles.
integrated/telemetry/	Unified trace, metric, and event emission.
integrated/replay/	Deterministic replay tools and run reconstruction helpers.
integrated/adapters/	Bridges to benchmark, memory, repository intelligence, and external release targets.
integrated/tests/	End-to-end, replay, contract, and gate tests for the whole system.
config/	Release profiles, environment matrices, and operational thresholds.
schemas/	Versioned JSON/YAML schemas for run, stage, evidence, and gate objects.

Folder rule: if a module changes the meaning of a run, it belongs in integrated/contracts or integrated/runtime. If it only consumes meaning, it belongs in an adapter or test package.

6. Data Contracts

The integrated system is only as trustworthy as its contracts. Contracts are written first, versioned, and treated as immutable once released.

Core contracts

Contract	Required fields	Behavioral rule
RunEnvelope	run_id, task_id, requester, environment, revision, policy_profile, seed	Must be unique and traceable from start to finish.
StageResult	run_id, stage_name, status, inputs, outputs, artifacts, timings	Every stage must emit one canonical result.
EvidenceBundle	run_id, provenance, artifact_refs, metric_refs, baseline_refs, approvals	Must preserve original evidence and derived evidence separately.
GateDecision	gate_id, run_id, policy_id, verdict, rationale, required_actions	Must be explainable without accessing logs.
ReleaseOutcome	release_id, run_id, publication_target, status, rollback_ref	Must be immutable after publication.

Example state model:

RunState: CREATED -> PLANNED -> RESEARCHED -> CODED -> TESTED -> REVIEWED -> EVALUATED -> RECOVERED? -> BENCH

State changes must be monotonic and observable. The system may branch into recovery paths, but it may not silently skip required states. Missing state is not “close enough”; it is a failure condition.

Configuration contract summary: environment profile, stage enablement, approval thresholds, benchmark threshold IDs, replay seed policy, retention policy, and rollback policy. The release environment must be explicitly declared, not inferred from server folklore.

7. State Management

State should be split into transient state, durable run state, and derived evidence state.

State class	Owner	Persistence	Consumed by
Transient runtime state	Orchestrator	In-memory with checkpointing	Current stage and live adapters
Durable run state	Run store	Append-only persistent store	Audit, replay, dashboards, gates
Derived evidence state	Evidence builder	Artifact store plus index	Reviewers, release, benchmark, humans
Policy state	Approval manager	Versioned policy repository	Gate evaluator and release manager
Replay state	Replay engine	Serialized fixture bundle	Regression tests and reproduction tooling

Ownership rule: the component that creates a piece of state owns the schema for that state, but not necessarily the storage backend. For example, the benchmark harness owns its run contract; the evidence store owns persistence mechanics. This prevents accidental schema drift and backend coupling.

Future agents should consume state through explicit selectors and published bundles, not by scanning storage directly. A planner should not open the run database. A deployment gate should not parse raw worktree logs. They should read the relevant typed artifact and act on it.

The state model is designed for resumability. If the system stops after review, the resume point must be reconstructable from durable state plus artifact references. A restart should not require guesswork or human memory, because human memory is a famously unreliable data store.

8. Component Specifications

Task Intake and Identity Service

- Purpose: Accept external request and create a run identity.
- Inputs: request metadata, environment profile, policy profile
- Outputs: RunEnvelope and trace root
- Failure conditions: Duplicate identity, missing policy, unknown environment
- Future integration points: benchmark admission and approval lookup

Stage Orchestrator

- Purpose: Route the run through all mandatory stages and manage checkpoints.
- Inputs: RunEnvelope, prior StageResult, policy decisions
- Outputs: next StageResult, checkpoint, failure context
- Failure conditions: illegal transition, timeout, stage starvation
- Future integration points: replay engine and release manager

Evidence Aggregator

- Purpose: Collect and normalize artifacts, metrics, and provenance.
- Inputs: stage outputs, artifact refs, metric events
- Outputs: EvidenceBundle and index entries
- Failure conditions: missing provenance, unlinked artifact, schema mismatch
- Future integration points: benchmark harness and audit tools

Safety and Approval Manager

- Purpose: Evaluate human and policy gates before release.
- Inputs: GateDecision inputs, policy versions, benchmark verdicts
- Outputs: pass / warn / block / manual review signal
- Failure conditions: policy ambiguity, missing approval signal, stale threshold
- Future integration points: release manager and deployment targets

Release Manager

- Purpose: Publish approved outputs and capture rollback references.
- Inputs: approved run, target environment, release policy
- Outputs: ReleaseOutcome and rollback marker
- Failure conditions: partial publish, unverified artifact, rollback failure
- Future integration points: external deployment systems and notifications

Replay and Forensics Service

- Purpose: Reconstruct runs and investigate failures from immutable evidence.
- Inputs: run_id, evidence bundle, fixture bundle
- Outputs: replay report and forensic diff
- Failure conditions: lost artifact, non-deterministic replay, corrupted fixture
- Future integration points: validation suite and postmortems

9. Implementation Sequence

Step 1 - Freeze system contracts

Create the run, stage, evidence, gate, and release schemas before any orchestration code exists. Without this, every module invents its own truth.

- Expected output: Versioned contracts, validation utilities, canonical examples.

Step 2 - Build run identity and lifecycle storage

A system cannot be managed if it cannot name its own runs. This step establishes durable identifiers and append-only run state.

- Expected output: Run store, identity service, lifecycle checkpoints.

Step 3 - Build the stage orchestrator

Route runs through the fixed stage pipeline and record every transition.

- Expected output: Stage dispatcher, transition guardrails, checkpoint manager.

Step 4 - Add evidence aggregation

Centralize artifact provenance so every output is traceable back to a stage and run.

- Expected output: Evidence bundle builder, artifact index, provenance chain.

Step 5 - Add gate evaluation

Apply policy and benchmark decisions before any release action is allowed.

- Expected output: Policy engine, gate evaluator, decision logging.

Step 6 - Add release publishing and rollback markers

Publish only after approval, and record enough evidence to roll back cleanly.

- Expected output: Release manager, rollback manifest, publication receipts.

Step 7 - Add replay and forensic tooling

Make the system explain itself after failures and prove determinism across restarts.

- Expected output: Replay engine, diff reports, fixture packs.

Step 8 - Add full-stack regression suites

Protect the integrated contract from accidental drift as the lower layers evolve.

- Expected output: End-to-end tests, golden runs, gate cases.

10. Validation Strategy

The validation strategy is system-level first. The goal is to prove not only that each piece works, but that the whole pipeline produces the same answer under the same inputs and clearly different answers under different inputs.

Test layer	What it proves	Pass condition
Contract tests	Every schema, state transition, and artifact shape is valid	Validators accept canonical fixtures and reject malformed inputs
Integration tests	The full pipeline executes end-to-end	A synthetic run reaches a final outcome with complete evidence
Replay tests	Deterministic reconstruction works	A replay yields the same verdict class and artifact hashes within policy
Policy tests	Approval and release rules are enforced	Unsafe runs are blocked, safe runs proceed, ambiguous runs are escalated
Failure-injection tests	Recovery paths work and do not corrupt evidence	Injected failures produce recoverable, traceable outcomes
Load tests	Concurrency and batching do not break isolation	Parallel runs preserve identity and artifact boundaries
Validation rule: if the integrated system can only be trusted when everyone watches it live, it is not validated. It is theatrical.		

11. Deliverables

Artifact	Description
integrated/runtime package	Run lifecycle, stage routing, and checkpoint support.
integrated/contracts package	Versioned contracts for system-wide data flow.
integrated/evidence package	Provenance, artifact indexing, and evidence bundles.
integrated/policy package	Approval and release gate logic.
integrated/replay package	Deterministic reproduction and forensic tooling.
integrated/telemetry package	Unified events, traces, and metrics for the whole system.
schemas bundle	Versioned JSON/YAML schema definitions.
config profiles	Environment-specific release and execution profiles.
regression fixture suite	Goldens for success, warning, failure, and recovery cases.
operator documentation	Runbook for release, recovery, and forensics.

If an artifact does not survive a restart, a replay, and a review by a different engineer, it is not a deliverable. It is a temporary message in a temporary folder pretending to be a system.

12. Acceptance Criteria

Criterion	Verification method
Every run is assigned a unique immutable identity.	Start two concurrent runs and verify no collisions or mixed artifacts.
Every mandatory stage emits a canonical StageResult.	Schema validation on a full synthetic run.
Recovery paths preserve provenance and do not overwrite evidence.	Inject a stage failure and confirm derived artifacts are separate from original evidence.
Approval cannot be bypassed by execution components.	Policy test attempts unauthorized release and must block.
Replay produces the same verdict class for the same frozen inputs.	Replay fixture comparison and artifact hash checks.
Benchmark and approval artifacts are linked to the same run_id.	End-to-end integration test verifies shared identity across all outputs.
A failed stage is explainable from recorded evidence alone.	Forensic test reconstructs the failure without parsing raw logs.

Completion means a release manager, an engineer, and an automated gate can inspect the same run and reach the same decision using the same artifacts. If they disagree, the system is not integrated; it is merely adjacent.

13. Common Failure Modes

- Letting stage modules write directly to each other instead of passing typed results through the orchestrator.
- Allowing the benchmark harness or approval logic to mutate run evidence.
- Treating a partial run as complete because most outputs were generated.
- Using logs as the primary data model instead of as a debugging view.
- Creating circular dependencies between policy, evidence, and execution layers.
- Burying recovery logic inside a stage instead of making it a first-class orchestrated path.
- Coupling deployment code to the success of one specific tool or agent implementation.

The biggest design trap is convenience. A shortcut that bypasses evidence or policy now will become a permanent blind spot later. The second biggest trap is “works on my run.” Integrated systems must be judged on repeatability, not anecdote.

14. Future Integration

- Part 6 contributes stable test outputs that become part of release evidence.
- Part 7 contributes evaluation judgments that influence acceptance and rejection.
- Part 8 contributes recoverability and controlled retry semantics.
- Part 9 contributes repository intelligence for run analysis and release impact.
- Part 10 contributes structure-aware reasoning over code relationships.
- Part 11 contributes context compression evidence for quality and token efficiency.
- Part 12 contributes multi-agent coordination telemetry and overhead analysis.
- Part 13 contributes benchmark verdicts and regression thresholds.
- Part 14 consumes the integrated evidence bundle to make release decisions.

Integration contract: future parts should only consume published artifacts and policies from Part 15, not internal runtime details. That keeps the whole platform evolvable instead of welded together with assumptions.

15. Completion Checklist

- Run identity is immutable and traceable through every stage.
- All mandatory stages have typed inputs and outputs.
- Evidence bundles preserve provenance and immutability.
- Approval and release gates are enforced by policy, not by convention.
- Rollback markers exist for approved releases.
- Replay and forensic tooling reproduce historical runs.
- Integration tests cover success, warning, failure, and recovery paths.
- Parallel runs preserve isolation and do not share artifacts.
- The system can answer who changed what, when, why, and under which evidence set.
- The final integrated system meets near-Codex behavior criteria for autonomy, control, and auditability.

Completion means the entire stack can be run, observed, questioned, replayed, and safely promoted as one coherent system. At that point the platform is no longer a pile of subsystems with ambitions. It is an engineering product.